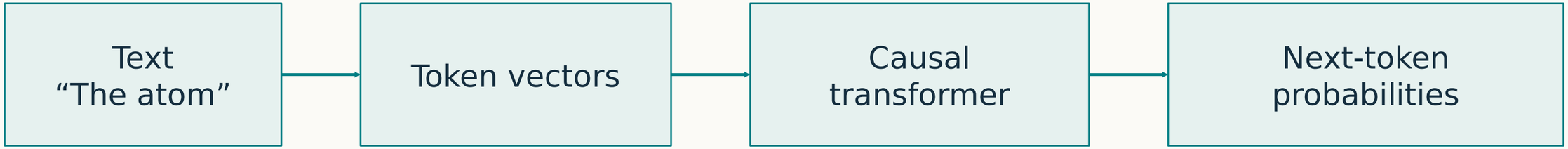
The decoder-only transformer

Mathematics, mechanisms, and a working language model

An undergraduate tutorial



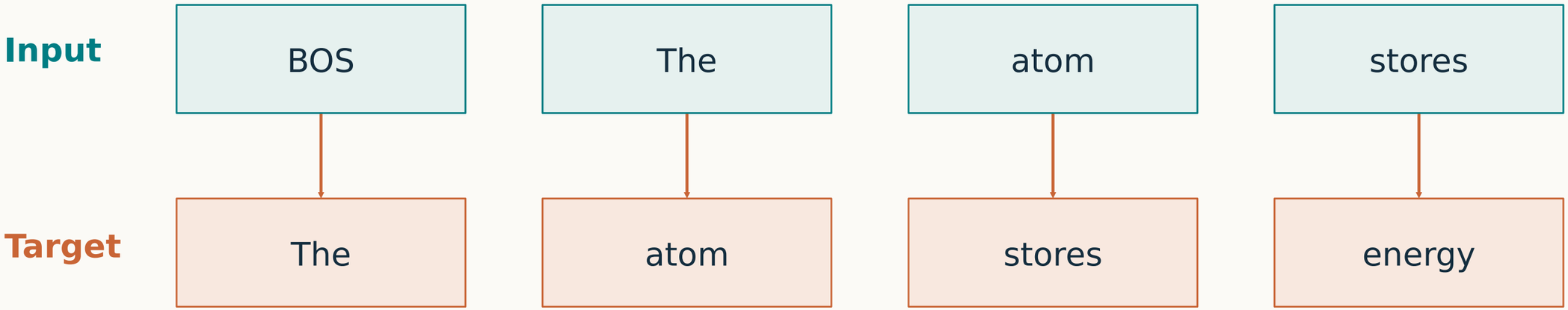
A small, dense GPT-style implementation in PyTorch

Prerequisites: vectors, matrix multiplication, derivatives, and basic Python

The next-token learning problem

Every position predicts the token immediately after it

$$p_\theta(x_1, \ldots, x_T \mid x_0) = \prod_{t=0}^{T-1} p_\theta(x_{t+1} \mid x_0, \ldots, x_t)$$

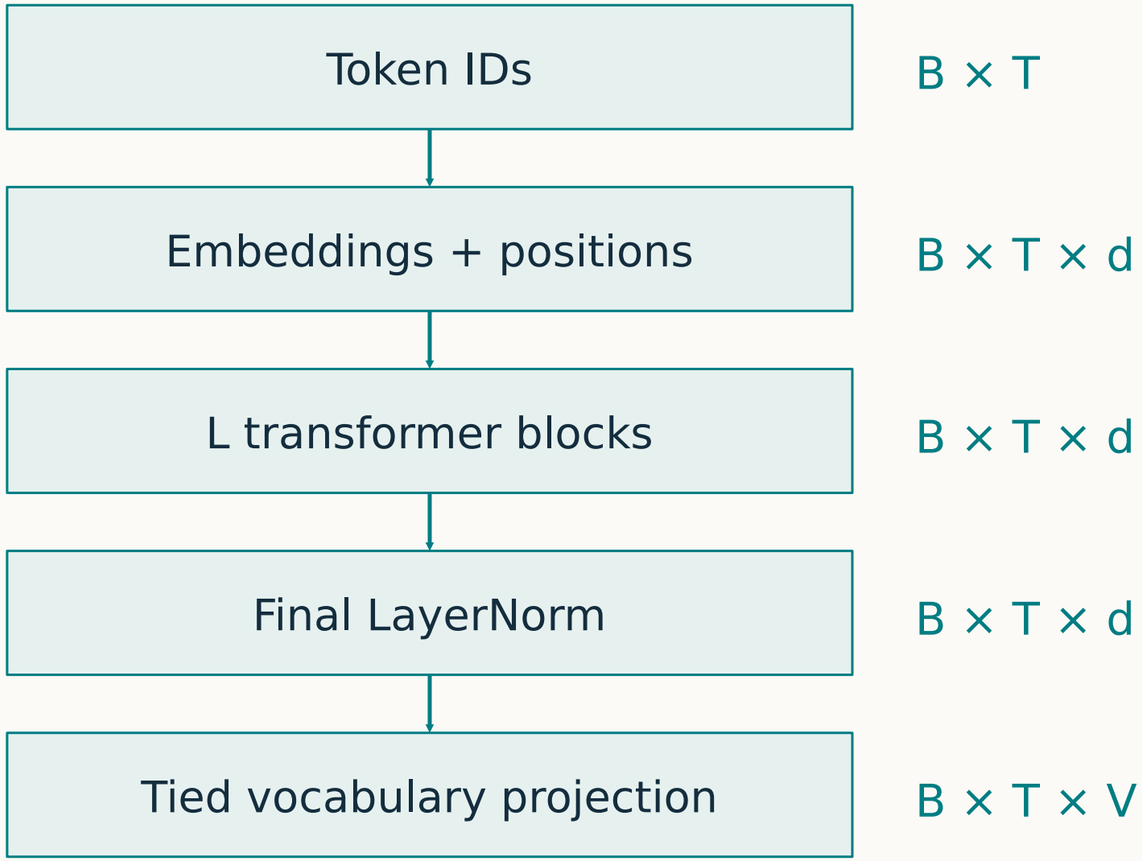


Training computes all positions together. The causal mask prevents access to the answer.

Word-sized tokens illustrate the shift here. The runnable model uses UTF-8 bytes.

The complete model and its tensor shapes

B sequences, T token positions, d features, H attention heads, V vocabulary entries



Symbol	Classroom run
V	258
T	64
d	64
H; d_h	4; 16
L; FFN width	2; 256

One residual stream keeps the same width d through every block.

Tokenization: text becomes integer IDs

The tokenizer defines the symbols the model can read and predict

Exact byte example



Text	Byte IDs
café	99, 97, 102, 195, 169
BOS / EOS	256 / 257

A byte is not always a character. “é” uses two bytes.

Subword BPE idea

```
l   o   w   e   r
↓
lo  w   e   r
↓
low e   r
↓
low er
```

Learn frequent adjacent-pair merges on training text, then freeze them.

Embedding lookup and position

Each token ID selects a learned vector, then position adds an order signal

$$X_t^{(0)} = E[x_t, :] + P[t, :], \qquad E \in \mathbb{R}^{V \times d}, \quad P \in \mathbb{R}^{T_{\max} \times d}$$

Toy ID	Feature 1	Feature 2	Feature 3
0	0.10	0.20	−0.10
1	0.50	−0.30	0.80
2	0.00	0.40	0.20
3	0.70	0.10	−0.20

ID 2 at position 0

```
E[2] = [0.00, 0.40, 0.20]
P[0] = [0.01,−0.02, 0.03]
X[0] = [0.01, 0.38, 0.23]
```

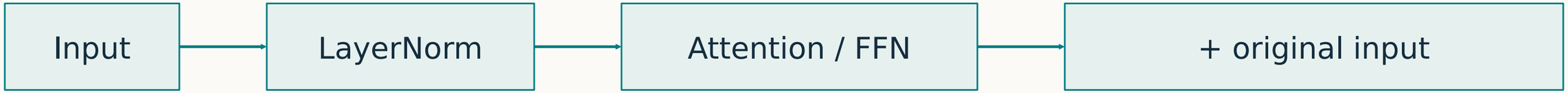
The same token at another position receives a different input vector.

The residual block and LayerNorm

Normalize features within each token, compute an update, and add it back

$$U = X + \mathrm{MHA}(\mathrm{LN}_1(X))$$

$$X_{\mathrm{next}} = U + \mathrm{FFN}(\mathrm{LN}_2(U))$$



$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \quad \mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$

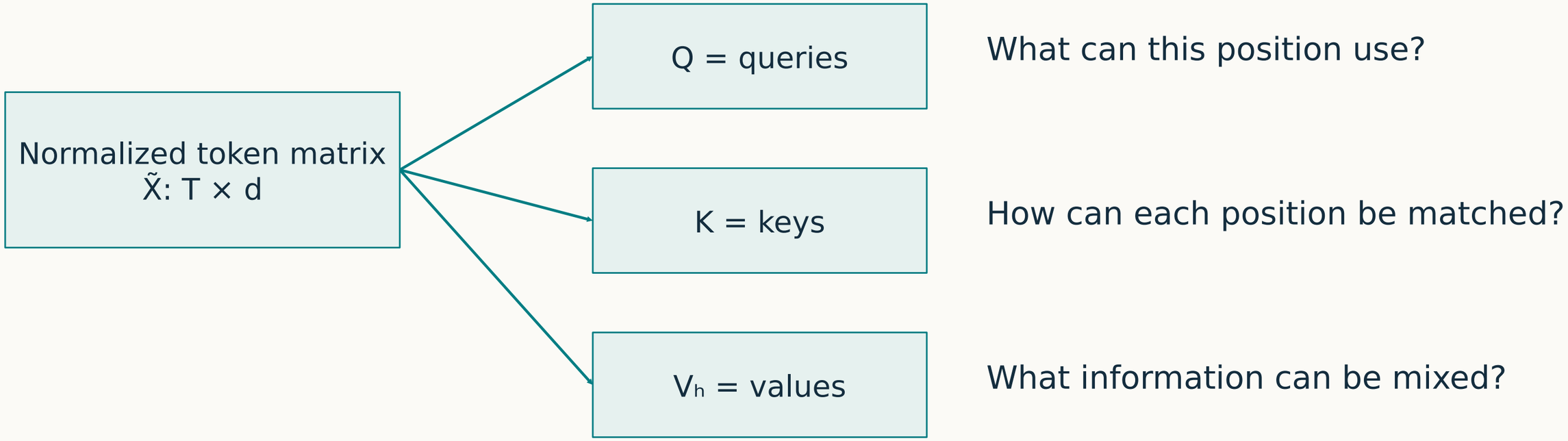
Example: [1, 2, 3] becomes approximately [−1.225, 0, 1.225] when γ = 1 and β = 0.

Each LayerNorm has its own learned scale γ and shift β. ε prevents division by zero.

Queries, keys, and values

Three learned projections give each token different roles in attention

$$Q = \tilde{X}W_Q + b_Q, \qquad K = \tilde{X}W_K + b_K, \qquad V_h = \tilde{X}W_V + b_V$$



One head: WQ, WK, WV each have shape d × d_h. The projected arrays have shape T × d_h.

Attention scores and the causal mask

The dot product measures compatibility, and the mask removes future positions

$$S = \frac{QK^{\mathsf{T}}}{\sqrt{d_h}} + M, \qquad M_{tj} = \begin{cases} 0 & j \le t \\ -\infty & j > t \end{cases}$$

query / key	0	1	2	3
0	0	−∞	−∞	−∞
1	0	0	−∞	−∞
2	0	0	0	−∞
3	0	0	0	0

Why divide by √dₕ?

A dot product adds d_h terms. Under a unit-variance independence assumption, its variance grows with d_h.

Example: $d_h = 2$, $q = [\sqrt{2}, 0]$

Keys: [2, 0], [1, 1], [0, 2]
Scaled scores: [2, 1, 0]

Softmax converts scores into weights

Exponentiate, then normalize each query row across the key positions

$$A_{tj} = \frac{\exp(S_{tj} - m_t)}{\sum_{k=0}^{T-1} \exp(S_{tk} - m_t)}, \qquad m_t = \max_k S_{tk}$$

Allowed score	Subtract max = 2	Exponential	Normalized weight
2	0	1.000	0.665
1	−1	0.368	0.245
0	−2	0.135	0.090

The weights are nonnegative and sum to 1.

Subtracting the row maximum leaves the answer unchanged and avoids overflow.

Attention mixes the value vectors

A weighted sum produces a new feature vector for each query position

$$Z = AV_h, \qquad z_t = \sum_{j=0}^{t} A_{tj} v_j$$

0.665 × [1, 0] + 0.245 × [0, 2] + 0.090 × [3, 1]

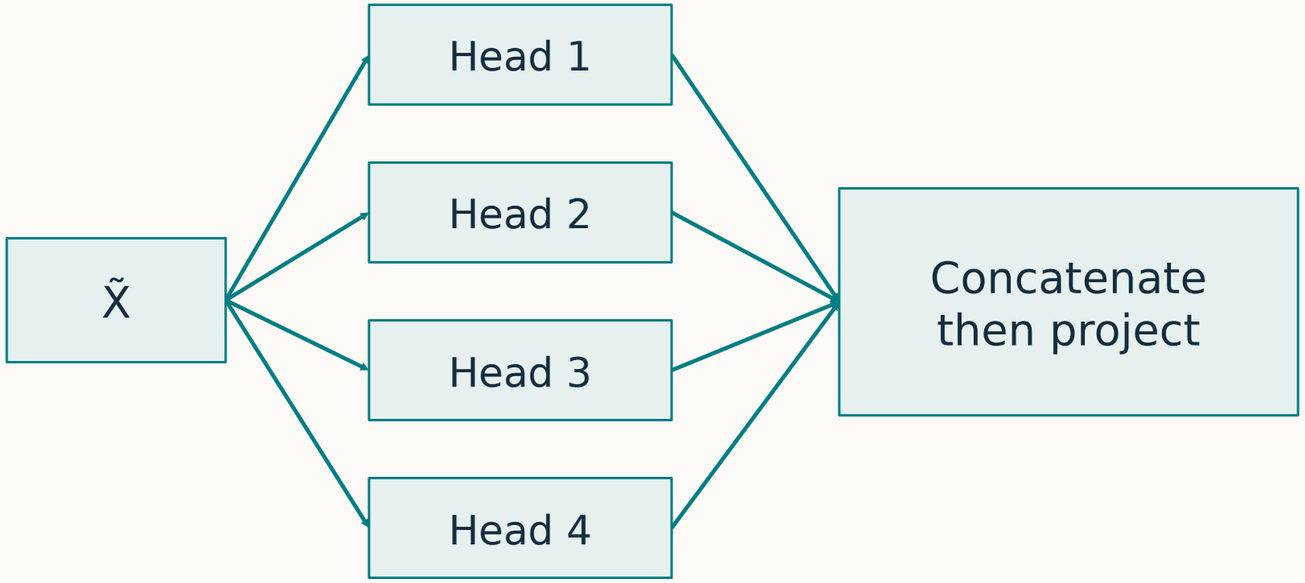
$$z = 0.66524[1, 0] + 0.24473[0, 2] + 0.09003[3, 1] \approx [0.935,\ 0.579]$$

The output has d_h features. The T attention weights control where information comes from.

Multiple heads and the attention implementation

Each head learns its own projections and its own attention distribution

$$\mathrm{MHA}(X) = \mathrm{Concat}(Z_1, \ldots, Z_H)W_O + b_O, \qquad d_h = d/H$$




```
q, k, v = split_heads(qkv(x))
s = (q @ k.transpose(-2, -1))
s = s / math.sqrt(head_dim)
s = s.masked_fill(~mask, -inf)
a = softmax(s, dim=-1)
z = a @ v
y = out_proj(merge_heads(z))
```

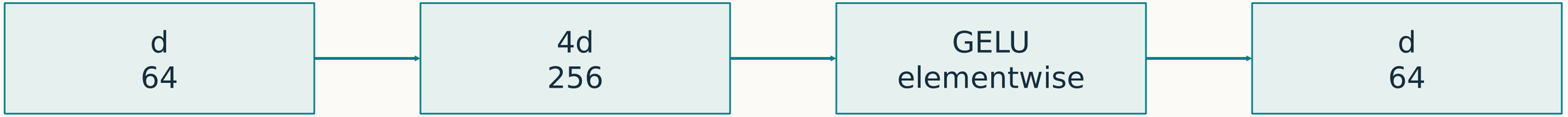
Shapes: Q, K, V are [B, H, T, d_h]

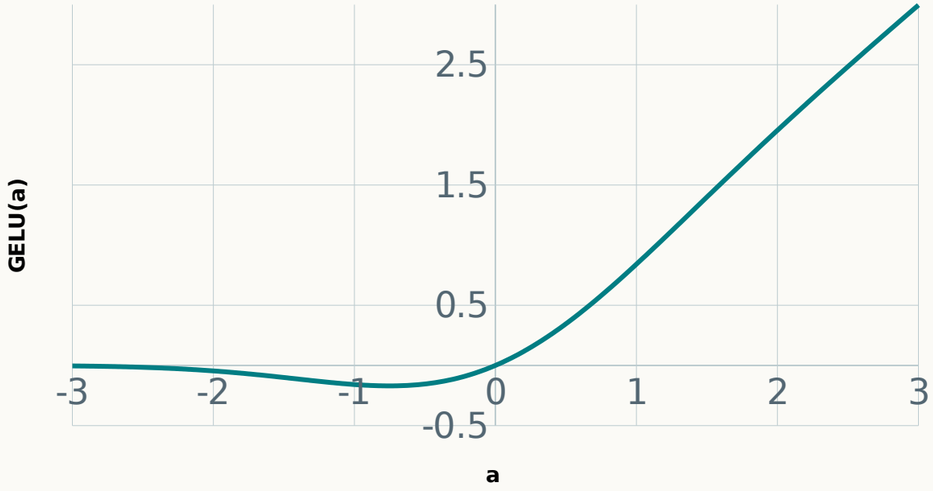
4 heads × 16 features = 64 features

The feedforward network and GELU

Attention mixes positions. The FFN transforms features independently at each position.

$$\mathrm{FFN}(u) = \mathrm{GELU}(uW_1 + b_1)W_2 + b_2$$





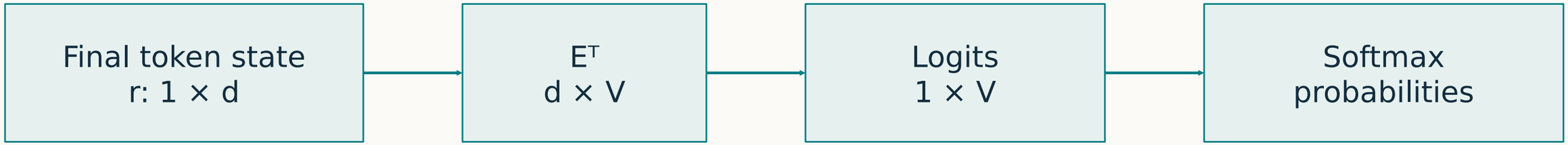
$$\mathrm{GELU}(a) = a\Phi(a), \qquad \Phi(a) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{a} e^{-s^2/2}\, ds$$

GELU(−1) ≈ −0.159
GELU(0) = 0
GELU(1) ≈ 0.841

The vocabulary head turns features into logits

A logit is an unnormalized score for one possible next token

$$R = \mathrm{LN}_f(X^{(L)}), \qquad \ell = RE^{\mathsf{T}}, \qquad p_{t,v} = \frac{e^{\ell_{t,v}}}{\sum_{u=0}^{V-1} e^{\ell_{t,u}}}$$



Weight tying reuses the input embedding table E for the output projection.

Training scores all positions. Generation uses only the final position’s logits.

Cross-entropy and backpropagation

Training increases the probability of the observed next token

$$\mathcal{L} = -\frac{1}{BT}\sum_{b=1}^{B}\sum_{t=0}^{T-1}\log p_{b,t,y_{b,t}}, \qquad y_{b,t} = x_{b,t+1}$$



$$\frac{\partial \mathcal{L}}{\partial \ell_{b,t,v}} = \frac{p_{b,t,v} - \mathbf{1}[v = y_{b,t}]}{BT}$$

For p = [0.665, 0.245, 0.090] and target 0: loss = −log(0.665) ≈ 0.408 nats.

Pass raw logits to cross_entropy. It performs log-softmax internally.

AdamW updates the learned parameters

Momentum smooths gradients, adaptive scaling adjusts steps, and weight decay shrinks matrices

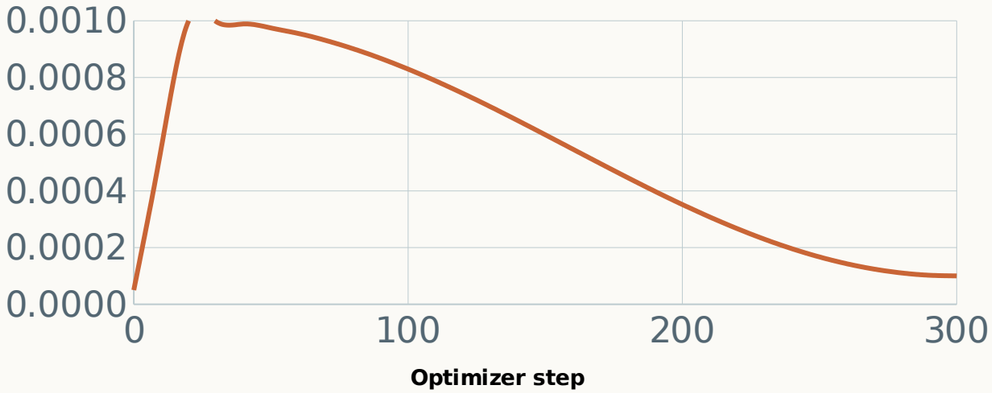
$$m_s = \beta_1 m_{s-1} + (1-\beta_1) g_s \qquad \widehat{m}_s = m_s / (1-\beta_1^s)$$

$$v_s = \beta_2 v_{s-1} + (1-\beta_2) g_s^2 \qquad \widehat{v}_s = v_s / (1-\beta_2^s)$$

$$\theta_{s+1} = (1-\eta_s \lambda)\theta_s - \eta_s \frac{\widehat{m}_s}{\sqrt{\widehat{v}_s} + \epsilon}$$

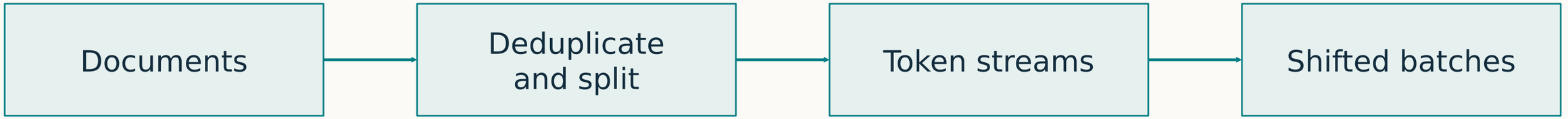
g_s is the gradient after global norm clipping at 1.0. Squares and division are elementwise.

Choice	Classroom setting
β_1, β_2	0.9, 0.95
ε; matrix weight decay	10^{-8}; 0.1
Peak learning rate	0.001
Warmup / total steps	20 / 300



The training harness

A complete experiment needs data boundaries, evaluation, and recoverable state




```
optimizer.zero_grad(set_to_none=True)
for microbatch in range(accum):
    logits, loss = model(x, y)
    (loss / accum).backward()
clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
```

Effective batch = batch_size × accumulation steps

Validation

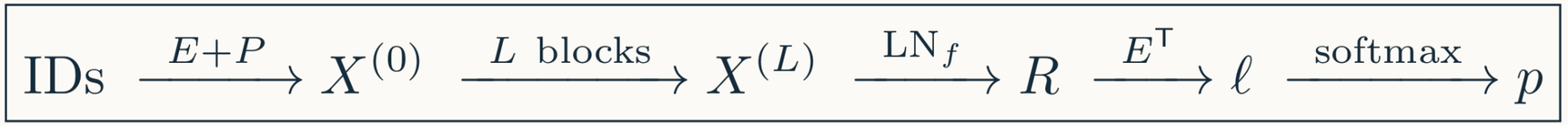
model.eval() + no_grad()
Fixed evaluation windows
Loss, perplexity, accuracy

Checkpoint

Weights, optimizer, step, configuration, and RNG states

The entire forward pass in code

The executable implementation keeps the mathematics visible



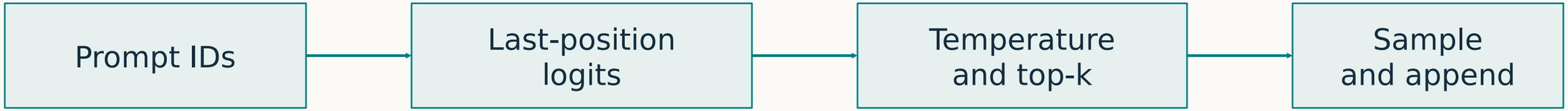

```
positions = torch.arange(T, device=ids.device)
x = token_embedding(ids) + position_embedding(positions)
for block in blocks:
    x = block(x)
x = final_norm(x)
logits = F.linear(x, token_embedding.weight)
loss = F.cross_entropy(
    logits.reshape(B*T, V), targets.reshape(B*T))
```

Each block: x = x + attention(ln1(x)), then x = x + ffn(ln2(x)).

Generating a token and continuing the sequence

The model predicts a distribution. A decoding rule chooses what to append.

$$p_v^{(\tau)} = \frac{\exp(\ell_v/\tau)}{\sum_u \exp(\ell_u/\tau)}, \qquad x_{\text{new}} \sim \text{Categorical}(p^{(\tau)})$$



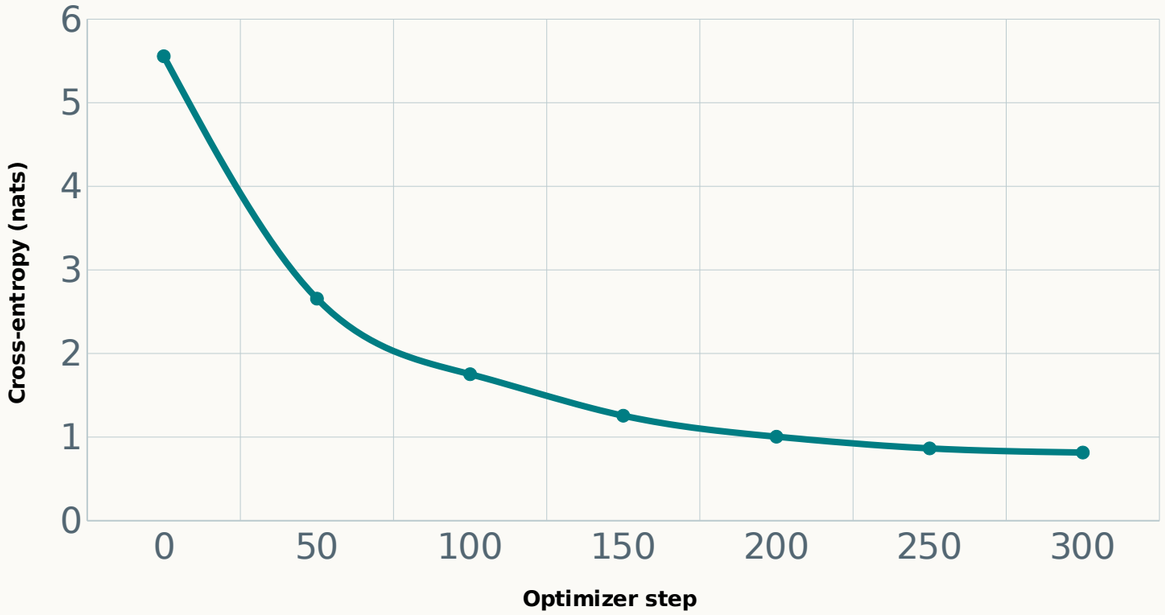
Temperature τ	p_0	p_1	p_2
0.5	0.867	0.117	0.016
1.0	0.665	0.245	0.090
2.0	0.506	0.307	0.186

```
logits = model(ids)[0][:, -1]
probs = softmax(logits / tau)
next_id = multinomial(probs, 1)
ids = cat([ids, next_id], dim=1)
```

Greedy decoding takes argmax. EOS or a length limit stops generation.

Running and checking the tutorial

A measured CPU run on an original synthetic science corpus



120,704 parameters
5.56 → 0.81 held-out loss
2.26 byte-token perplexity

Checks: causal independence, normalized attention, manual/SDPA agreement, gradients, and learning.

```
python corpus.py
python train.py --steps 300 --context 64 --width 64 --layers 2 \
  --heads 4 --batch-size 8 --lr 0.001 --out runs/classroom
python generate.py --checkpoint runs/classroom/best.pt
```

References and a reading path

Primary sources for the architecture, mathematics, and implementation

1 **Vaswani et al. · Attention Is All You Need**
2017 · arxiv.org/abs/1706.03762

2 **Radford et al. · Language Models are Unsupervised Multitask Learners**
2019 · github.com/openai/gpt-2

3 **Sennrich et al. · Neural Machine Translation of Rare Words with Subword Units**
2016 · arxiv.org/abs/1508.07909

4 **Ba et al. · Layer Normalization**
2016 · arxiv.org/abs/1607.06450

5 **Hendrycks & Gimpel · Gaussian Error Linear Units**
2016 · arxiv.org/abs/1606.08415

6 **Loshchilov & Hutter · Decoupled Weight Decay Regularization**
2019 · arxiv.org/abs/1711.05101

7 **PyTorch · Linear, LayerNorm, GELU, CrossEntropyLoss, AdamW, SDPA**
docs.pytorch.org · API details and numerical conventions

Read 2 for the decoder architecture, 1 for attention, and 6 for the optimizer.